BUBBLE SORT

```
#include<stdio.h>
#include<conio.h>
void print_arr(int arr[],int n)
{
  int i;
  printf("\n\n array elements :");
  for(i=0;i<n;i++)
  {
       printf(" %d , ",arr[i]);
  }
}
void sort_arr(int arr[],int n)
{
   int i,j,temp;
   for(i=0;i<n-1;i++)
   {
       for(j=i+1;j<n;j++)
       {
              if(arr[i]>arr[j])
              {
                 temp = arr[i];
                 arr[i] = arr[j];
                 arr[j] = temp;
              }
```

```
        }
      print_arr(arr,n);
    }
}
void main()
{
    int arr[10],n,i;
    clrscr();
    printf("\n Enter size of array :  ");
    scanf("%d",&n);
    printf("\n enter elements : ");
    for(i=0;i<n;i++)
    {
          scanf("%d",&arr[i]);
    }
    printf("\n Before sorting array elements : \n");
    print_arr(arr,n);
    printf("\n after sorting:");
    sort_arr(arr,n);
    getch();
}
```

****

INSERTION SORT

```
#include<stdio.h>
#include<conio.h>
void print_arr(int arr[],int n)
{
  int i;
  printf("\n\n Array Elements are : ");
  for(i=0;i<n;i++)
  {
     printf("%d , ",arr[i]);
  }
}
void insertion_sort(int arr[],int n)
{
   int i,j,key;
   for(i=1;i<n;i++)
   {
        key = arr[i];
        for(j = i -1 ;j>=0 && arr[j]>key;j--)
        {
            arr[j+1] = arr[j];
            print_arr(arr,n);
        }
        arr[j+1] = key;
        printf("\n\n Disaplaying arrar elements after interatioon : %d",i+1);
```

```
        print_arr(arr,n);
    }
}
void main()
{
    int arr[5],n,i;
    clrscr();
    printf("Enter size of array : ");
    scanf("%d",&n);
    printf("Enter array elements : ");
    for(i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }
    printf("\n Array elements before sorting : ");
    print_arr(arr,n);
    insertion_sort(arr,n);
    getch();

}
```

SELECTION SORT

```
#include<stdio.h>
#include<conio.h>
void print_arr(int arr[],int n)
{
  int i;
  printf("\n\n Array Elements are : ");
  for(i=0;i<n;i++)
  {
     printf("%d , ",arr[i]);
  }
}
void selection_sort(int arr[],int n)
{
   int i,j,min,index,temp;
   for(i=0;i<n-1;i++)
   {
        index = i;
        printf("\n Interation : %d ",i+1);
        min = i+1;
        for(j=i+1;j<n;j++)
        {
           if(arr[min] > arr[j])
           {
                min = j;
```

```
            }
            printf("\n min index = %d",min);
        }
        //printf("\n Interation :  %d ",i);
        printf("\n arr[%d] = %d",index,arr[index]);
        printf("\n arr[%d] = %d",min,arr[min]);
        if(arr[index] > arr[min])
        {
            temp = arr[index];
            arr[index] = arr[min];
            arr[min] = temp;
        }
        print_arr(arr,n);
    }
}
void main()
{
    int arr[5],n,i;
    clrscr();
    printf("Enter size of array : ");
    scanf("%d",&n);
    printf("Enter array elements : ");
    for(i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
```

```
    }
    printf("\n Array elements before sorting : ");
    print_arr(arr,n);
    selection_sort(arr,n);
    getch();

}
```

****

LINEAR SEARCH

```
#include<stdio.h>
#include<conio.h>
int linear_search(int arr[],int n,int num)
{
   int i;
   for(i=0;i<n;i++)
   {
        if(arr[i]==num)
        {
                return i;
        }
   }
   return -1;
}
void main()
{
        int arr[10],num,i,n,ans;
        clrscr();
        printf("enter input size");
        scanf("%d",&n);
        printf("Enter array elements : ");
        for(i=0;i<n;i++)
        {
                scanf("%d",&arr[i]);
```

```
        }
        printf("Enter element for searching");
        scanf("%d",&num);
        ans = linear_search(arr,n,num);
        if(ans==-1)
        {
            printf("element not found");
        }
        else
        {
            printf("your value found at location %d",ans);
        }
        getch();
}
```

****

BINARY SEARCH (ITERATIVE APPROACH)

```
#include<stdio.h>
#include<conio.h>
int binary_search(int arr, int n, int key)
{
  int beg=0;
  int end = n-1;
  int mid = 0;
  while(beg<=end)
        {
              mid = (beg+end)/2;
              if arr[mid]==key
              {
                return  mid;
               }
              else if (arr[mid] > key)
              {
                    end = mid -1;
               }
           else
           {
              beg = mid + 1;
              }
```

```
        }
  return -1;
}
void main()
{
    int n,arr[5],key, ans , i;
    clrscr();
        printf("Enter size of array : ");
        scanf("%d",&n);
        printf("Enter array elements : ");
        for(i=0;i<n;i++)
        {
            scanf("%d",&arr[i]);
        }
        printf("\n Enter key value : ");
        scanf("%d",&key);
        ans = binary_search(arr,n,key);
        if(ans==-1)
        {
            printf("\n Element Not Found");
        }
        else
        {
```

```
        printf("\n Element  Found at location :  %d",ans);
    }
    getch();
}
```

****

BINARY SEARCH (RECURSIVE APPROACH)

```
#include<stdio.h>
#include<conio.h>
int arr[10],n; //global variable
int binary_search(int beg,int end,int key)
{
   int mid;
   if(beg==end)
       {
          if(arr[beg]==key)
               {
                  return beg;
               }
               else
               {
                   return -1;
               }
       }
       else
       {
          mid = (beg+end)/2;
               if(arr[mid]==key)
               {
```

```
            return mid;
        }
        else if(key<arr[mid])
        {
            return binary_search(beg,mid-1,key);
        }
        else
        {
            return binary_search(mid+1,end,key);
        }
    }
}
void main()
{
  int i,key,ans;
  clrscr();
  printf("Enter size of array : ");
  scanf("%d",&n);
  printf("Enter array elements : ");
  for(i=0;i<n;i++)
  {
     scanf("%d",&arr[i]);
  }
```

```
  printf("\n Enter key value for searching : ");
  scanf("%d",&key);
  ans = binary_search(0,n-1,key);
  if(ans==-1)
  {
    printf("\n Value not found");
  }
  else
  {
    printf("\n Value found on location : %d",ans);
  }
  getch();
}
```

QUICK SORT

```
#include<stdio.h>
#include<conio.h>
int arr[10],n;
int partition (int low, int high)
{
	int pivot = arr[low]; // pivot element
	int i = low;
	int j = high;
	int temp;
	//printf("\n Value of low = %d",low);
	//printf("\n value of high = %d",high);
	while(i<=j)
	{
		while(arr[i]<=pivot)
		{
		i++;
		}
		while(arr[j]>pivot)
		{
		j--;
		}
		if(i<=j)
```

```
            {
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            }
        }
        arr[low] = arr[j];
        arr[j] = pivot;
        return j;
}

void print_arr()
{
        int i;
        for (i = 0; i < n; i++)
        {
        printf("%d ", arr[i]);
        }
        printf("\n");
}

void quick_sort(int low, int high)
{
```

```
        if (low < high )
        {
                int p = partition(low, high);
                quick_sort(low, p - 1);
                quick_sort(p + 1, high);
        }
}
void main()
{
 int i;
        clrscr();
    printf("\n Enter size of array :  ");
    scanf("%d",&n);
    printf("\n enter elements : ");
    for(i=0;i<n;i++)
    {
         scanf("%d",&arr[i]);
    }
    printf("\n Before sorting array elements : \n");
    print_arr();
    //printf("\n after sorting:");
    quick_sort(0,n-1);
    printf("\n after sorting array elements : \n");
```

```
    print_arr();
    getch();
}
```

MERGE SORT

```
#include<stdio.h>
#include<conio.h>
int arr[10],n;
void  merge (int low,int mid, int high)
{
        int ls[10],rs[10];
        int n1 = mid - low + 1;
        int n2 = high - mid;
        int i,j,k;
        //passing data in left subarray
        for(i=0;i<n1;i++)
        {
           ls[i] = arr[low+i];
        }
        //passing data in right subarray
        for(j=0;j<n2;j++)
        {
           rs[j] = arr[mid+1+j];
        }
        i=0;
        j=0;
        k=low;
```

```
while(i<n1 && j<n2)
{
  if(ls[i]<rs[j])
  {
        arr[k] = ls[i];
        i++;
        k++;
  }
  else
  {
        arr[k] = rs[j];
        j++;
        k++;
  }
}
while(i<n1)
{
        arr[k] = ls[i];
        i++;
        k++;
}
while(j<n2)
{
```

```
            arr[k] = rs[j];
            j++;
            k++;
        }

}

void print_arr()
{
        int i;
        for (i = 0; i < n; i++)
        {
        printf("%d ", arr[i]);
        }
        printf("\n");
}

void mergeSort(int beg, int end)
{
        if (beg < end)
        {
            int mid = (beg + end) / 2;
            mergeSort(beg, mid);
```

```
            mergeSort(mid + 1, end);
            merge(beg, mid, end);
        }
}

void main()
{
 int i;
        clrscr();
    printf("\n Enter size of array :  ");
    scanf("%d",&n);
    printf("\n enter elements : ");
    for(i=0;i<n;i++)
    {
         scanf("%d",&arr[i]);
    }
    printf("\n Before sorting array elements : \n");
    print_arr();
    //printf("\n after sorting:");
    mergeSort(0,n-1);
    printf("\n after sorting array elements : \n");
    print_arr();
    getch();
```

```
}
```

ACTIVITY SELECTION PROBLEM

```
#include<stdio.h>
#include<conio.h>
int act[20],st[20],et[20],sel_act[20],n,count=0;
void asp()
{
  int i,lsal;
  for(i=0;i<n;i++)
  {
        if(i==0)
        {
           sel_act[count] = act[i];
                lsal = i;
                count++;

        }
        else
        {
           if(st[i]>=et[lsal])
                {
                  sel_act[count] = act[i];
                        lsal = i;
                        count++;
```

```
                }
        }
  }
  printf("\n Selected activities are : \n");
  for(i=0;i<count;i++)
  {
     printf("\n activity %d",sel_act[i]);
  }
}
void main()
{
        int i,j,temp;
  clrscr();
  printf("Enter number of activities  : ");
  scanf("%d",&n);
  for(i=0;i<n;i++)
  {
    act[i] = i+1;
    printf("Enter start and end value for activity %d \n",i+1);
         scanf("%d %d",&st[i],&et[i]);
  }
  //sort data in terms of end time in asceding order
  for(i=0;i<n-1;i++)
```

```
  {
    for(j=i+1;j<n;j++)
        {
           if(et[i]>et[j])
                {
                  temp = et[i];
                       et[i] = et[j];
                       et[j] = temp;

                       temp = st[i];
                       st[i] = st[j];
                       st[j] = temp;

                       temp = act[i];
                       act[i] = act[j];
                       act[j] = temp;
                }
        }
  }
  asp();
  getch();
}
```

FRACTIONAL KNAPSACK PROBLEM

```
#include<stdio.h>
#include<conio.h>
float x[10],weight[10],profit[10],ratio[10],capacity;
int n;
void knapsack()
{
   int i;
      float tp=0,rc=capacity;
      for(i=0;i<n;i++)
      {
         if(weight[i]>rc)
            {
               break;
            }
            else
            {
               rc = rc - weight[i];
                  tp = tp + profit[i];
```

```
            }
        }
        if(i<n)
        {
            tp = tp + ((rc / weight[i] ) * profit[i])
        }
        printf("\n Profit = %f ",tp);

}
void main()
{
     int i,j;
         float temp;
    clrscr();
        printf("Enter number of objects : ");
        scanf("%d",&n);
        printf("\n Enter values of weight and profit : ");
        for(i=0;i<n;i++)
        {
            printf("\n Enter value for object %d :  \n",i+1);
```

```
    scanf(" %f  %f ",&weight[i],&profit[i]);
  }
  printf("Enter value of capacity : ");
  scanf("%f",&capacity);
  for(i=0;i<n;i++)
  {
    ratio[i] = profit[i]/weight[i];
  }
  //sort data in terms of ratio(profit/weight) in
descending order
  for(i=0;i<n-1;i++)
  {
    for(j=i+1;j<n;j++)
      {
        if(ratio[i]<ratio[j])
          {
            temp = ratio[i];
              ratio[i] = ratio[j];
              ratio[j] = temp;
```

```
                    temp = weight[i];
                    weight[i] = weight[j];
                    weight[j] = temp;

                    temp = profit[i];
                    profit[i] = profit[j];
                    profit[j] = temp;
                }
            }
        }
        knapsack();
        getch();
}
```

PRIMS ALGORITHM (MCST)

```
#include<stdio.h>
#include<conio.h>
void prims_algo(int a[][10],int n)
{
   int ne=1,sum=0,i,j,x=0,y=0,min;
      int t[10][10],selected[10];
      for(i=1;i<=n;i++)
      {
        selected[i]=0;
      }
      for(i=1;i<=n;i++)
      {
        for(j=1;j<=n;j++)
            {
                  t[i][j] = 0;
            }
      }
      selected[1]= 1;
```

```
while(ne<n)
{
   min = 1234;
   for(i=1;i<=n;i++)
   {
      if(selected[i]==1)
         {
            for(j=1;j<=n;j++)
               {
                  if(selected[j]==0 && a[i][j]!=0)
                     {
                        if(min>a[i][j])
                           {
                              min = a[i][j];
                              x = i;
                              y = j;
                           }
                     }
               }
         }
```

```
        }
        t[x][y] = min;
        selected[y] = 1;
        sum = sum + t[x][y];
        printf("\n selected edge : (%d,%d) =
%d",x,y,t[x][y]);
        ne = ne + 1;
      }
      printf("\n Minimum cost = %d",sum);

}
void main()
{
  int a[10][10],n,edges,weight,start,end,i,j;
  clrscr();
  printf("Enter number of vertices : ");
  scanf("%d",&n);
  printf("Enter number of edges : ");
  scanf("%d",&edges);
  for(i=0;i<=n;i++)
```

```
{
   for(j=0;j<=n;j++)
      {
         a[i][j] = 0;
      }
}
for(i=1;i<=edges;i++)
{
    printf("\n Enter start of edge : ");
       scanf("%d",&start);
       printf("\n Enter end of edge : ");
       scanf("%d",&end);
       printf("\n Enter weight : ");
       scanf("%d",&weight);
       a[start][end] = weight;
       a[end][start] = weight;
}
printf("\n Adjancecy matrix : \n");
for(i=1;i<=n;i++)
{
```

```
    for(j=1;j<=n;j++)
        {
           printf(" %d ",a[i][j]);
        }
        printf("\n");
  }
  prims_algo(a,n);
  getch();
}
```

JOB SCHEDULING PROBLEM

==========

```
#include<stdio.h>
#include<conio.h>
int jid[10],profit[10],deadline[10],max=0;
int allocated[10],sum=0,n,filledslots = 0;
int sel_jobs[10];
void job_scheduling()
{
 int i,j,val;
   for(i=0;i<max;i++)
   {
      allocated[i] = 0;
   }
   for(i=0;i<n && filledslots!=max;i++)
   {

            val = deadline[i]-1;
            if(allocated[val]!=1)
               {
```

```
   allocated[val]=1;
      sel_jobs[filledslots] = jid[i];
      filledslots++;
      sum = sum + profit[i];

}
else
{
  for(j=max-1;j>=0;j--)
     {
        if(allocated[j]!=1)
           {
              allocated[val]=1;
                 sel_jobs[filledslots] = jid[i];
                 filledslots++;
                 sum = sum + profit[i];
           }
     }
}
```

```
   }
   printf("\n Selected jobs : ");
   for(i=0;i<filledslots;i++)
   {
      printf(" %d , ",sel_jobs[i]);
   }
   printf("\n profit : %d ",sum);
}
void main()
{
  int i,j,temp;
  clrscr();
  printf("Enter number of jobs : ");
  scanf("%d",&n);
  for(i=0;i<n;i++)
  {
   printf("\n Enter data for %d ",i+1);
     jid[i] = i+1;
        printf(" \n Enter profit : ");
     scanf("%d",&profit[i]);
```

```
      printf(" \n Enter deadline in hours : ");
   scanf("%d",&deadline[i]);
}
// sort data in terms of profit in descedning order
// find the maximum value in deadline
max = deadline[0];
for(i=0;i<n-1;i++)
{
      for(j=i+1;j<n;j++)
      {
        if(profit[i]<profit[j])
        {
           printf("\n profit[%d] = %d",i,profit[i]);
           printf("\n profit[%d] = %d ",j,profit[j]);

               temp = profit[i];
               profit[i] = profit[j];
               profit[j] = temp;

               temp = deadline[i];
```

```
            deadline[i] = deadline[j];
            deadline[j] = temp;

            temp = jid[i];
            jid[i] = jid[j];
            jid[j] = temp;
        }
    }
    if(max < deadline[i])
    {
        max = deadline[i];
    }

}
    printf("Entered data is : ");
    for(i=0;i<n;i++)
    {
        printf("\n\n job id : %d",jid[i]);
            printf("\n profit : %d",profit[i]);
            printf("\n deadline: %d",deadline[i]);
```

```
        }
        printf("\n Maximum value in deadline = %d",max);
        job_scheduling();
  getch();
}
```